Trabalho 2 - Resolução de problemas com busca

Inteligência Artificial - 2025.2

Dezembro de 2025

1 Introdução

Este trabalho tem como objetivo implementar, comparar e experimentar na prática como os algoritmos clássicos de busca descritos no Capítulo 3 de Russell & Norvig se comportam quando aplicados à resolução do Problema do Caixeiro Viajante (TSP), permitindo analisar suas diferenças estruturais e empíricas. São estudadas as abordagens de:

- Busca em Largura (BFS)
- Busca em Profundidade (DFS)
- Busca de Custo Uniforme (UCS)
- Busca Gulosa
- Algoritmo A*

O problema consiste em encontrar o circuito de menor custo em um grafo completo com pesos positivos. As estratégias de busca serão comparadas em termos de qualidade da solução, tempo de execução e quantidade de nós gerados e visitados.

2 Código-fonte, estrutura do projeto e instruções de uso

2.1 Código-fonte

Para evitar problemas de incompatibilidade, como diferenças entre pacotes, versões do *Python* ou qualquer outro fator que possa impactar a avaliação do trabalho, optou-se pela utilização do *Google Colab*. O código pode ser acessado por meio do seguinte link: `https://colab.research.google.com/drive/1tkWtiuOOBsRZ7gDBTPE13JTz9hOHe_mS?usp=sharing`. É importante salientar que é necessário estar conectado a uma conta do Google para executar os algoritmos.

2.2 Estrutura do projeto

Para tornar o desenvolvimento mais organizado e facilitar tanto a execução quanto a avaliação do trabalho, o notebook no *Google Colab* foi estruturado em quatro seções principais:

- **Modelagem** - Esta seção contém o arquivo `estado.py`, responsável por definir a estrutura fundamental usada pelos algoritmos de busca para representar um *estado* no Problema do Caixeiro Viajante (TSP). Discutiremos a modelagem com mais detalhes na seção 3.

- **Algoritmos implementados** - Reúne as implementações dos algoritmos estudados. Abaixo a listagem dos algoritmos implementados e seus respectivos nomes de arquivos.
  - `dfs.py` (Busca em Profundidade);
  - `bfs.py` (Busca em Largura);
  - `ucs.py` (Busca de Custo Uniforme);
  - `gulosa.py` (Busca Gulosa)
  - `aestrela.py` (Algoritmo A*)

- **Gerador de grafos** - Contém as funções responsáveis pela criação dos grafos utilizados nos testes. O arquivo correspondente a essa função é denominado como `grafo.py`.
  - `grafo.py`: Responsável por gerar, de forma aleatória, a matriz de distâncias entre as cidades, com valores positivos variando de 0 a 100. Para garantir que os experimentos sejam reprodutíveis e que cada algoritmo seja testado sob exatamente as mesmas condições, utilizamos uma *semente* (*seed*) no gerador de números aleatórios.

    A semente é um valor numérico que inicializa o gerador de aleatoriedade. Quando fixamos uma semente, o gerador produz sempre a mesma sequência de números aleatórios, permitindo que diferentes execuções e diferentes algoritmos utilizem a mesma instância do grafo. Caso desejado, a semente pode ser alterada ou desativada, conforme explicado na subseção de instruções de uso.

    A seguir, apresentamos um exemplo de matriz de distâncias gerada para o caso de 6 cidades:

$$\begin{bmatrix} 0 & 82 & 15 & 4 & 95 & 36 \\ 82 & 0 & 32 & 29 & 18 & 95 \\ 15 & 32 & 0 & 14 & 87 & 95 \\ 4 & 29 & 14 & 0 & 70 & 12 \\ 95 & 18 & 87 & 70 & 0 & 76 \\ 36 & 95 & 95 & 12 & 76 & 0 \end{bmatrix}$$

Tabela 1: Exemplo de matriz de distâncias gerada aleatoriamente para um grafo com 6 cidades. Cada entrada grafo[i][j] representa o custo para viajar da cidade i para a cidade j.

- **Execução e comparativos entre algoritmos** - Nesta seção são realizados os testes práticos, a execução dos métodos e a comparação dos resultados obtidos.

- **Gráficos** — Nesta seção, todos os algoritmos são comparados entre si e, ao final, são gerados gráficos que ilustram os resultados dessa comparação.

A utilização da diretiva `%%writefile` possibilita que cada componente (algoritmos, geradores ou módulos auxiliares) seja salvo como um arquivo independente diretamente no ambiente do Colab. Essa abordagem torna o código mais modular, facilita a organização do projeto e permite importar cada parte conforme necessário durante as execuções.

2.3 Instruções de uso

Na apresentação dos resultados, optou-se por adotar duas estratégias complementares, organizadas em seções distintas.

A primeira estratégia, denominada *Execução e comparativos entre algoritmos*, consiste na execução individual de cada método de busca. Nessa seção, o algoritmo a ser avaliado é selecionado explicitamente, permitindo uma análise detalhada de seu comportamento isolado, incluindo métricas como custo da solução, tempo de execução, número de nós gerados e número de nós visitados.

A segunda estratégia corresponde à seção de *Gráficos*, na qual todos os algoritmos são comparados simultaneamente. Nessa etapa, os resultados obtidos nas execuções individuais são consolidados e utilizados para gerar gráficos comparativos, possibilitando uma visualização global do desempenho relativo entre os métodos avaliados. Esses gráficos permitem analisar, de forma integrada, o impacto do crescimento do número de cidades sobre a qualidade das soluções e a eficiência computacional de cada algoritmo.

2.4 Execucão e comparativos entre algoritmos

2.4.1 Observação importante

Todos os testes foram executados em um ambiente com **47 GB de RAM** e utilizando uma **TPU v5e-1**. Ao executar este notebook em sua própria conta do Google Colab, o Google poderá impor limitações de recursos (RAM, CPU ou outros), o que pode afetar o tempo de execução ou inviabilizar testes muito grandes. Caso esteja disponível na sua conta, siga o passo a passo abaixo.

Antes de executar os experimentos, é necessário configurar corretamente o ambiente de execução no Google Colab. Para isso, siga os passos abaixo:

1. Na barra superior do Colab, clique na opção *Runtime*.
2. No menu exibido, selecione a opção *Change runtime type*.
3. Na janela de configuração, mantenha o *Runtime type* como *Python 3*.
4. Em *Hardware accelerator*, selecione a opção *v5e-1 TPU*.
5. Verifique se a opção *Runtime version* está definida como *Latest (recommended)*.
6. Por fim, clique no botão *Save* para aplicar as configurações.

Após a conclusão desses passos, o ambiente estará corretamente configurado e pronto para a execução das células do notebook.

2.4.2 Execução de algoritmos separadamente

Antes de executar o código, é importante definir os **parâmetros iniciais** utilizados nos testes. Esses parâmetros estão localizados na seção *"Execução e comparativos entre algoritmos"*. Procure pelo trecho identificado como `configurações iniciais`.
Nessa parte, você pode ajustar:

- o número de cidades a serem utilizadas nos experimentos;
- o uso ou não de uma semente (`seed`) para geração aleatória, caso não deseje utilizar a semente, atribua `None`;

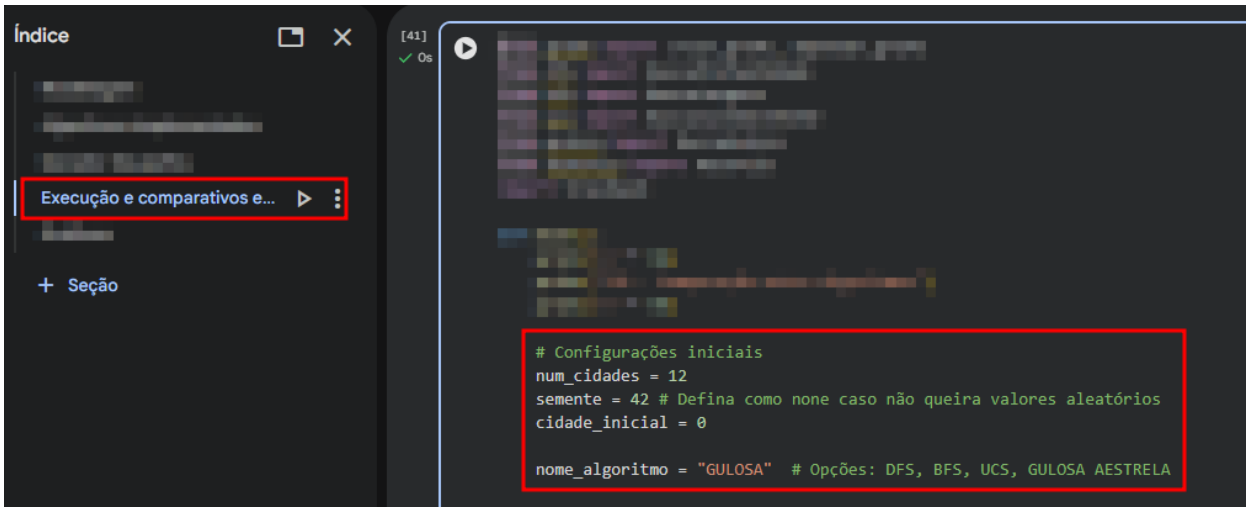

Figura 1: Configuração inicial das cidades, semente. ponto de partida e algoritmo.

- a cidade inicial, que por padrão é definida como `0`.
- o nome do algoritmo a ser utilizado na execução, podendo assumir os valores: DFS, BFS, UCS, GULOSA ou AESTRELA

```
num_cidades = 10
semente = 42
cidade_inicial = 0
# Opções: DFS, BFS, UCS, GULOSA AESTRELA
nome_algoritmo = "GULOSA"
```

Após realizar todos os Após a realização dos ajustes iniciais, basta executar as células do notebook na ordem em que aparecem, utilizando o ícone de execução disponível em cada seção. Esse procedimento garante o carregamento de todos os arquivos e dependências necessários.

Concluída essa etapa, a célula da seção *Execução e comparativos entre algoritmos* pode ser executada para observar a execução dos algoritmos e a coleta dos resultados experimentais. Em caso de alterações nas configurações, é suficiente reexecutar apenas essa célula.

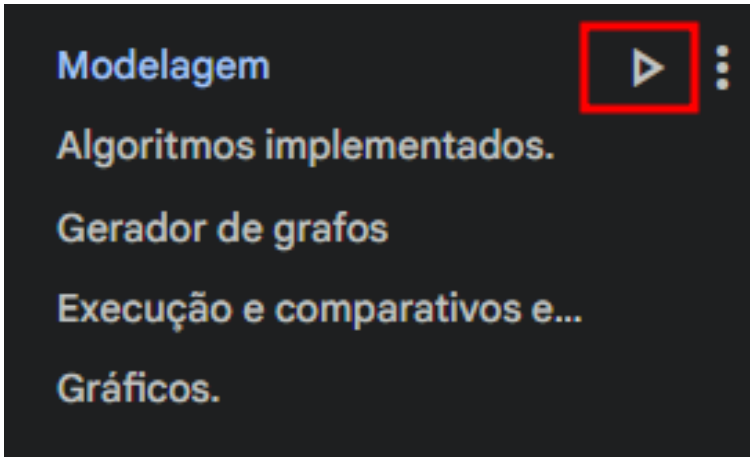

Figura 2: Executando células necessárias.

2.4.3 Resultados gráficos

Ao acessar a seção *Gráficos*, mantêm-se necessário as mesmas configurações citadas anteriores, com exceção da escolha do algoritmo a ser executado. Nessa etapa, todos os algoritmos são executados simultaneamente e, ao final, são gerados gráficos comparativos entre eles.

3 Modelagem do problema

3.1 Representação de estados

A estrutura central utilizada para representar o progresso da busca no Problema do Caixeiro Viajante foi a classe `Estado`. Essa classe funciona como um nó da árvore (ou grafo) de busca, encapsulando todas as informações necessárias para descrever a situação atual da execução do algoritmo.

Durante a modelagem, surgiram desafios relacionados ao consumo de memória, o que nos levou a empregar técnicas como bitmask, uma forma compacta de armazenar conjuntos de elementos usando bits individuais para indicar se cada item está presente ou não.

Formalmente, o espaço de estados adotado neste trabalho é definido como

$$S = \{\, (c, V) \mid c \in \{0, \ldots, N-1\},\ V \subseteq \{0, \ldots, N-1\},\ c \in V \,\},$$

onde c representa a cidade atual do agente e V representa o conjunto de cidades já visitadas. Na implementação, o conjunto V é codificado por meio de uma *bitmask* inteira, em que cada bit indica se uma cidade foi visitada (1) ou não (0).

A restrição $c \in V$ reflete o fato de que, no TSP, o agente só pode estar fisicamente em uma cidade que já tenha sido marcada como visitada. O estado inicial é, portanto,

$$s_0 = (c_{\text{inicial}}, \{c_{\text{inicial}}\}),$$

com sua respectiva bitmask inicial.
Para ilustrar a codificação utilizada, considere um problema com $N = 6$ cidades (índices 0 a 5). A representação do conjunto de cidades visitadas é feita através de uma *bitmask*, onde cada bit funciona como uma chave:

$$\text{bit } i = \begin{cases} \mathbf{1}, & \text{se a cidade } i \text{ foi visitada,} \\ \mathbf{0}, & \text{caso contrário.} \end{cases}$$

O valor decimal dessa máscara é calculado somando-se as potências de 2 correspondentes aos bits ativos. Matematicamente:

$$\text{Mascara} = \sum_{i=0}^{N-1} (\text{bit}_i \times 2^i)$$

Exemplo prático: Suponha que o agente esteja na cidade 3 e o conjunto de cidades já visitadas seja $V = \{0, 2, 3\}$. A construção do inteiro que representa este conjunto é:

Cidade (i)	5	4	**3**	**2**	1	**0**
Bit (b_i)	0	0	**1**	**1**	0	**1**
Valor (2^i)	32	16	**8**	**4**	2	**1**

Logo, o valor inteiro da máscara é a soma dos pesos onde o bit é 1:

$$\text{visitadas} = 8 + 4 + 1 = 13.$$

Portanto, o estado de busca é armazenado de forma compacta como:

$$s = (\text{cidade_atual} = 3,\ \text{visitadas} = 13).$$

Nesta representação, o número 13 é a assinatura única do subconjunto $\{0, 2, 3\}$. Isso permite verificar pertinência e realizar transições de estado utilizando operações binárias de baixo nível, que são computacionalmente mais eficientes que manipular listas ou vetores.

As principais operações utilizadas são:

- **Verificação de pertinência (Check):** Para saber se a cidade i já foi visitada, utiliza-se a operação *AND* lógico com um deslocamento de bits:

  $$\text{visitado} = (\text{máscara} \ \&\ (1 \ll i))$$

  Se o resultado for diferente de zero, a cidade consta no conjunto.

- **Transição de estado (Add):** Ao mover-se para uma nova cidade j, o novo estado é gerado pela operação *OR* lógico, que "liga"o bit correspondente sem alterar os demais:

  $$\text{nova_máscara} = \text{máscara} \mid (1 \ll j)$$

Essas instruções são executadas em tempo constante $O(1)$ diretamente pela CPU, o que garante desempenho superior em comparação à busca linear $O(N)$ em vetores ou listas tradicionais.

A arquitetura foi projetada com ênfase em eficiência de memória e desempenho, dado que o espaço de estados cresce exponencialmente com o número de cidades.

3.2 Atributos

Cada instância de `Estado` contém os seguintes campos essenciais:

- `cidade_atual`: inteiro representando o índice da cidade onde o agente se encontra. Esse atributo define o vértice atual do percurso parcial.

- `visitadas`: estrutura crítica da classe, representada como uma *bitmask* inteira. Cada bit indica se uma cidade foi visitada (1) ou não (0). Essa representação substitui estruturas mais custosas como `set` ou `frozenset`, reduzindo o uso de memória de maneira significativa.

- `pai`: referência para o estado imediatamente anterior, possibilitando reconstrução eficiente do caminho após o término da busca.

- `custo`: custo acumulado para alcançar o estado corrente.

3.3 Ações do agente

As ações disponíveis ao agente consistem na transição da cidade atual para qualquer cidade ainda não visitada. Em termos formais, no estado $s = (\text{cidade_atual}, \text{visitadas})$, o conjunto de ações é dado por:

$$A(s) = \{\, c \mid c \notin \text{visitadas} \,\}.$$

Cada ação produz um novo estado em que:

- a cidade atual passa a ser a cidade selecionada;
- a bitmask é atualizada;
- o custo acumulado é acrescido do custo da aresta correspondente;
- o estado atual é armazenado como `pai` para fins de reconstrução do caminho.

3.4 Função de custo

A função de custo de caminho corresponde à soma dos pesos das arestas percorridas. Ao atingir um estado objetivo, soma-se também o custo de retorno ao nó inicial.

3.5 Teste de objetivo

O teste de objetivo verifica se o agente já visitou **todas as cidades exatamente uma vez**. Note que o retorno à cidade inicial não faz parte da condição de objetivo; ele é adicionado apenas ao custo final após o estado terminal ser alcançado.

A verificação baseia-se na contagem dos bits ativados na bitmask associada ao estado, indicando o conjunto de cidades visitadas:

$$|V(s)| = N.$$

Uma vez satisfeita essa condição, o estado é considerado terminal, e a solução completa é obtida posteriormente conectando-se a última cidade visitada de volta à cidade de origem.

Formalmente, o predicado de objetivo é definido como:

$$\text{Objetivo}(s) = \begin{cases} \textbf{True}, & \text{se visitadas} = \text{bitmask_completo}, \\ \textbf{False}, & \text{caso contrário}. \end{cases}$$

A classe `Estado` implementa essa verificação por meio da função `ja_visitei_todas`, que compara o número de bits ligados com o total de cidades:

$$\textit{teste_objetivo}(s) \equiv \texttt{ja_visitei_todas}(s, N).$$

A eficiência do teste é crucial para algoritmos como A*, BFS e DFS, pois evita expansões desnecessárias e identifica rapidamente estados terminais graças à representação compacta em bitmask.

4 Algoritmos implementados

Nesta seção, descrevem-se os principais aspectos de cada algoritmo, incluindo a modelagem de estados, estruturas de dados utilizadas, critérios de expansão e métricas coletadas.

4.1 Busca em Profundidade (DFS)

A busca em profundidade (DFS) foi implementada utilizando uma pilha (estrutura LIFO) para controlar a ordem de expansão dos estados. Em cada iteração, o algoritmo remove o estado no topo da pilha, expande seus sucessores e insere todos eles novamente na própria pilha, preservando o comportamento de exploração profunda antes de retornar aos níveis iniciais.

A busca encerra quando a pilha esvazia ou quando todas as soluções viáveis foram encontradas, guardando sempre a de menor custo. Isso reduz significativamente o consumo de memória, mas permite que caminhos equivalentes sejam gerados repetidas vezes, aumentando substancialmente o número de nós produzidos. Essa ausência de poda torna o DFS particularmente vulnerável à explosão combinatória típica do TSP, já que a profundidade máxima da busca é igual ao número total de cidades.

Os estados utilizam uma representação compacta baseada em *bitmask* para o conjunto de cidades visitadas. Essa abordagem reduz drasticamente o custo de memória por nó e permite operações de expansão rápidas, como ativar bits correspondentes às cidades sucessoras ou consultar se uma cidade já foi visitada.

A expansão consiste em gerar um novo estado para cada cidade ainda não incluída no *bitmask*. Cada sucessor armazena a nova cidade atual, o *bitmask* atualizado, o ponteiro para o pai e o custo acumulado. Todos os nós gerados e expandidos são contabilizados para as métricas registradas ao final da execução.

O critério de parada local ocorre sempre que um estado contém todas as cidades no *bitmask*. Nesse momento, o algoritmo calcula o custo de retorno para a cidade inicial e atualiza a melhor solução global caso esse valor seja menor que o encontrado anteriormente. A busca encerra quando a pilha esvazia, momento em que o algoritmo terá explorado um vasto conjunto de caminhos parciais e completos, guardando sempre a solução de menor custo encontrada.

Em termos de eficiência, o DFS apresenta baixo uso de memória, mas tende a gerar e expandir um número muito elevado de estados, especialmente devido à ausência de heurísticas e mecanismos de poda. Apesar de completo e sempre capaz de retornar uma solução válida, ele não encontra necessariamente a solução ótima primeiro, e sua performance se deteriora rapidamente com o aumento do número de cidades.

4.2 Busca em Largura (BFS)

Diferente da abordagem em profundidade, a Busca em Largura (BFS) explora o espaço de estados nível por nível. A implementação utiliza uma fila FIFO (*First-In, First-Out*) para gerenciar a fronteira de busca. O algoritmo remove o estado mais antigo da fila, gera todos os seus sucessores e os insere no final da fila.

No contexto do Caixeiro Viajante, onde todas as soluções válidas encontram-se na mesma profundidade N (número de cidades), a BFS comporta-se necessariamente como uma busca exaustiva por camadas.

Assim como na DFS, adotou-se a modelagem *tree-like search* e a representação de estados via *bitmask*. Embora a BFS seja teoricamente intensiva em memória — pois precisa armazenar toda a fronteira da árvore, que cresce exponencialmente — a otimização com *bitmask* permitiu a execução de instâncias de até $N = 12$ cidades. Sem essa representação compacta, a BFS ultrapassaria rapidamente a capacidade de RAM disponível.

A BFS registra informações de execução, incluindo o número de nós gerados e expandidos, o tempo total de execução e o custo da melhor solução obtida. Quando um estado terminal é alcançado, isto é, quando todas as cidades foram visitadas, o algoritmo calcula o

custo da aresta de retorno à cidade inicial e atualiza a melhor solução caso um caminho de menor custo seja encontrado. Ao final, a reconstrução do caminho é realizada retrocedendo pelos ponteiros de pai armazenados em cada estado.

4.3 Busca de Custo Uniforme (UCS)

A Busca de Custo Uniforme (Uniform Cost Search — UCS) é um algoritmo de busca não informada que seleciona, a cada iteração, o estado com menor custo acumulado desde o estado inicial. Seu critério de avaliação considera exclusivamente o custo real do caminho percorrido, sendo definido por:

$$f(n) = g(n),$$

onde $g(n)$ representa o custo acumulado até o estado n.

A fronteira de busca é mantida em uma fila de prioridade (min-heap), ordenada pelo valor de $g(n)$. Em cada iteração, o algoritmo remove da fila o estado com menor custo acumulado e expande seus sucessores. Diferentemente da Busca em Largura (BFS), que expande estados por nível, a UCS expande estados em ordem crescente de custo, o que a torna adequada para grafos com custos heterogêneos.

Na implementação desenvolvida neste trabalho, a UCS utiliza uma estratégia de *graph search*, mantendo um dicionário de estados já alcançados (*reached*), indexados pela cidade atual e pela bitmask de cidades visitadas. Para cada estado, armazena-se o menor custo conhecido para alcançá-lo. Um estado só é expandido caso o caminho atual represente uma melhoria em relação aos custos previamente registrados, evitando expansões redundantes e aumentando a eficiência do algoritmo.

O problema do Caixeiro Viajante é modelado por meio de uma representação de estados baseada em *bitmask*, que identifica de forma compacta o conjunto de cidades já visitadas. A expansão de um estado gera sucessores correspondentes às cidades ainda não visitadas, atualizando o custo acumulado de acordo com a matriz de distâncias.

O teste de objetivo ocorre quando todas as cidades foram visitadas. Nesse momento, é computado o custo de retorno à cidade inicial. Devido à estratégia de expansão por custo crescente, a **primeira solução completa encontrada é garantidamente ótima**, permitindo a interrupção imediata da busca.

Desde que todos os custos das arestas sejam não negativos, a Busca de Custo Uniforme é **completa e ótima**, garantindo a obtenção da solução de menor custo global. Entretanto, por não utilizar nenhuma informação heurística para guiar a busca, a UCS ainda sofre com a explosão combinatória do espaço de estados, tornando-se rapidamente custosa em termos de tempo e memória à medida que o número de cidades aumenta. Por esse motivo, ela é utilizada neste trabalho principalmente como referência para comparação com os algoritmos de busca informada.

4.4 Busca Gulosa

A Busca Gulosa é um algoritmo de busca informada que seleciona, a cada iteração, o estado que aparenta estar mais próximo do objetivo segundo uma função heurística $h(n)$. Diferentemente da Busca de Custo Uniforme e do algoritmo A*, a Busca Gulosa **desconsidera completamente o custo acumulado** do caminho já percorrido. Sua função de avaliação é definida por:

$$f(n) = h(n).$$

No Problema do Caixeiro Viajante, essa estratégia leva o algoritmo a priorizar decisões localmente promissoras, sem levar em conta os custos já incorridos. Como consequência,

a Busca Gulosa tende a expandir poucos nós e apresentar tempos de execução reduzidos, porém não oferece garantias de otimalidade, podendo resultar em soluções de custo elevado.

A implementação adotada neste trabalho utiliza a mesma modelagem de estados baseada em *bitmask*, bem como a mesma definição de ações e função de custo empregadas pelos demais algoritmos. A fronteira de busca é mantida em uma fila de prioridade ordenada exclusivamente pelo valor heurístico $h(n)$. Em cada iteração, o estado com menor valor heurístico é removido da fila e expandido.

Nos experimentos apresentados neste trabalho, a Busca Gulosa foi executada utilizando uma heurística específica, cuja definição, propriedades e justificativa são descritas detalhadamente na Seção 5. O critério de parada ocorre quando um estado terminal é encontrado, isto é, quando todas as cidades foram visitadas. Nesse momento, o algoritmo computa o custo de retorno à cidade inicial e registra a solução obtida.

Os resultados experimentais mostram que, embora eficiente em termos de tempo, a Busca Gulosa apresenta desempenho inferior em qualidade de solução à medida que o tamanho da instância cresce, evidenciando seu caráter míope e local.

4.5 Algoritmo A*

O algoritmo A* é uma estratégia de busca informada que combina as ideias da Busca de Custo Uniforme e da Busca Gulosa, considerando simultaneamente o custo real acumulado do caminho e uma estimativa heurística do custo restante até o objetivo. Sua função de avaliação é definida como:

$$f(n) = g(n) + h(n),$$

onde $g(n)$ é o custo acumulado desde o estado inicial até o estado n, e $h(n)$ é uma função heurística que estima o custo mínimo restante para completar o percurso.

Quando a heurística utilizada é **admissível**, isto é, quando nunca superestima o custo real até o objetivo, o algoritmo A* é garantidamente **completo e ótimo**. Nessas condições, a primeira solução encontrada possui custo mínimo global.

Na implementação realizada neste trabalho, o A* utiliza a mesma modelagem de estados baseada em *bitmask* e a mesma função de custo adotada nos demais algoritmos. A fronteira de busca é gerenciada por uma fila de prioridade ordenada pelo valor de $f(n)$. Em cada iteração, o estado com menor valor de $f(n)$ é removido da fila e expandido.

Além disso, foi adotada uma estratégia de *graph search*, mantendo um registro dos estados já alcançados, indexados pela cidade atual e pela bitmask de cidades visitadas. Esse mecanismo permite descartar estados que tenham sido alcançados anteriormente com custo menor, evitando expansões redundantes e melhorando significativamente a eficiência do algoritmo.

A heurística empregada nos experimentos principais do algoritmo A*, bem como sua justificativa teórica e análise de admissibilidade, são apresentadas detalhadamente na Seção 5. Os resultados experimentais demonstram que essa escolha permite ao A* encontrar soluções ótimas expandindo significativamente menos nós do que a Busca de Custo Uniforme, especialmente para instâncias de maior dimensão.

5 Funções heurísticas propostas

Os algoritmos de busca informada avaliados neste trabalho, Busca Gulosa e Algoritmo A*, dependem diretamente de funções heurísticas para estimar o custo restante até a con-

clusão do percurso no Problema do Caixeiro Viajante. A qualidade dessas heurísticas impacta tanto a eficiência da busca quanto a qualidade das soluções encontradas.

Durante o desenvolvimento do trabalho, foram implementadas diferentes funções heurísticas com distintos níveis de informação e custo computacional. Entretanto, com o objetivo de manter a clareza, a comparabilidade dos resultados e a objetividade da análise experimental, optou-se por empregar apenas uma heurística representativa em cada algoritmo nos experimentos principais. As demais heurísticas foram utilizadas de forma exploratória e para validação do comportamento dos algoritmos.

A seguir, descrevem-se as heurísticas implementadas e sua utilização em cada algoritmo.

5.1 Heurística do vizinho mais próximo completo (Busca Gulosa)

Para a Busca Gulosa, foi adotada como heurística principal a estratégia do *vizinho mais próximo completo*. Essa heurística estima o custo restante a partir de um estado simulando a continuação do percurso: a cada passo, a cidade não visitada mais próxima é selecionada, até que todas as cidades sejam visitadas, finalizando com o retorno à cidade inicial.

Diferentemente de heurísticas locais mais simples, essa abordagem considera explicitamente todo o conjunto de cidades ainda não visitadas, fornecendo uma estimativa mais informada do custo restante. Por esse motivo, ela tende a produzir soluções de melhor qualidade quando comparada a heurísticas baseadas apenas em informações parciais do estado.

Apesar de ser computacionalmente mais custosa do que heurísticas triviais, seu uso na Busca Gulosa mostrou-se viável, uma vez que o algoritmo expande um número reduzido de nós.

É importante ressaltar que essa heurística **não é admissível**, pois pode superestimar o custo real restante. O algoritmo guloso do vizinho mais próximo tende a fazer escolhas localmente ótimas que podem resultar em caminhos globalmente sub-ótimos. Isso explica por que, apesar de eficiente, a Busca Gulosa não garante a otimalidade das soluções encontradas.

5.2 Heurística da arvore geradora mínima (Algoritmo A*)

No algoritmo A*, foi utilizada como heurística principal uma estimativa baseada na construção de uma Árvore Geradora Mínima (MST) sobre o conjunto de cidades ainda não visitadas. A estimativa heurística é composta pelo custo da MST, somado à menor aresta ligando a cidade atual a esse conjunto e à menor aresta de retorno à cidade inicial.

Essa heurística é admissível, pois o custo de uma MST representa um limite inferior para qualquer circuito que conecte os mesmos vértices. Formalmente, qualquer tour completo deve conter todas as arestas que conectam as cidades não visitadas, e a MST fornece o menor custo possível para essa conexão. Como um tour forma um ciclo e a MST forma apenas uma árvore (sem ciclos), o custo do tour será necessariamente maior ou igual ao custo da MST. Consequentemente, essa heurística nunca superestima o custo real necessário para completar o percurso, garantindo as propriedades de completude e otimalidade do algoritmo A*.

Os resultados experimentais confirmam que essa heurística fornece um bom equilíbrio entre custo computacional e poder informativo, permitindo ao A* encontrar soluções ótimas expandindo significativamente menos nós do que a Busca de Custo Uniforme, especialmente em instâncias com maior número de cidades.

5.3 Outras Heurísticas Implementadas

Além das heurísticas utilizadas nos experimentos principais, foram implementadas outras funções heurísticas com menor grau de informação, tais como heurísticas baseadas na menor aresta incidente a cada cidade não visitada e heurísticas triviais do tipo $h(n) = 0$, utilizadas como linha de base.

Essas heurísticas foram empregadas apenas para fins de validação, comparação conceitual e análise do impacto do grau de informação heurística no comportamento dos algoritmos, não sendo incluídas nas tabelas principais de resultados.

6 Resultados e comparações

Antes de apresentar os resultados obtidos, é importante definir o cenário dos experimentos. Para fins didáticos e para permitir uma comparação justa entre os algoritmos, utilizamos instâncias do problema do caixeiro viajante (TSP) com diferentes tamanhos, variando o número de cidades conforme descrito a seguir:

$$\mathbf{N = 6,\ 8,\ 10,\ 12}$$

Cada conjunto de testes foi executado sempre partindo da mesma cidade inicial (nó 0) e utilizando a mesma matriz de custos para todos os algoritmos. Essa padronização garante que as diferenças observadas nos resultados, como tempo de execução, custo da solução e número de nós gerados, sejam exclusivamente atribuídas ao comportamento dos algoritmos, e não a variações nos dados de entrada.

Além disso, todas as matrizes foram geradas utilizando a mesma *semente aleatória* (`seed`), garantindo que os valores das arestas fossem idênticos em todas as execuções. O uso de uma semente fixa é fundamental para assegurar a reprodutibilidade dos experimentos: sem ela, cada execução poderia produzir pesos diferentes para as arestas, tornando impossível comparar corretamente o desempenho entre algoritmos ou entre diferentes tamanhos de N.

Caso matrizes distintas ou sementes diferentes fossem utilizadas, os resultados deixariam de ser comparáveis, pois variações no custo total poderiam refletir apenas modificações acidentais na estrutura do grafo. Portanto, a uniformidade das matrizes e o uso de uma semente fixa são elementos essenciais para uma análise justa, rigorosa e reprodutível.

6.1 Tabelas de resultados

Os resultados obtidos para cada algoritmo foram organizados em tabelas separadas de acordo com o número de cidades da instância. Em cada tabela são apresentados o custo da melhor solução encontrada, o tempo total de execução, o número de nós gerados e o número de nós visitados durante o processo de busca.

Essa organização por tamanho de instância possibilita observar de forma clara como o crescimento do número de cidades N afeta o desempenho de cada método, tanto em termos do custo das soluções quanto da eficiência computacional. As tabelas a seguir apresentam os resultados obtidos para instâncias com $N = 6$, 8, 10 e 12 cidades.

6.1.1 Resultados para 6 cidades

Algoritmo	Custo	Tempo (s)	Nós gerados	Nós visitados
BFS	157	0,0046	325	206
DFS	157	0,0024	325	206
UCS	157	0,0004	165	76
Gulosa	157	0,0002	15	5
A*	157	0,0009	29	9

6.1.2 Resultados para 8 cidades

Algoritmo	Custo	Tempo (s)	Nós gerados	Nós visitados
BFS	127	0,0645	13.699	8.660
DFS	127	0,1127	13.699	8.660
UCS	127	0,0029	1351	442
Gulosa	290	0.0003	28	7
A*	127	0,0145	116	28

6.1.3 Resultados para 10 cidades

Algoritmo	Custo	Tempo (s)	Nós gerados	Nós visitados
BFS	144	3,5522	986.409	623.530
DFS	144	13,9879	986.409	623.530
UCS	144	0,0194	9.225	2.296
Gulosa	224	0,0006	45	9
A*	144	0,0124	328	61

6.1.4 Resultados para 12 cidades

Algoritmo	Custo	Tempo (s)	Nós gerados	Nós visitados
BFS	182	300,3449	108.505.111	68.588.312
DFS	182	228,4736	108.505.111	68.588.312
UCS	182	0.1139	56.331	11.254
Gulosa	377	0,0011	66	11
A*	182	0,3506	3669	590

6.2 Gráficos

Com o objetivo de facilitar a análise comparativa do desempenho dos algoritmos avaliados, foram gerados gráficos a partir dos dados experimentais utilizando a biblioteca `matplotlib`. Cada visualização relaciona o número de cidades N com métricas relevantes, como tempo de execução, número de nós gerados e custo da solução encontrada, permitindo observar tendências de crescimento, escalabilidade e diferenças entre as estratégias de busca.

Os resultados evidenciam o impacto da explosão combinatória do espaço de estados, especialmente nas buscas não informadas, que apresentam comportamento exponencial à medida que N aumenta. Em contrapartida, observa-se que o uso de heurísticas contribui significativamente para a redução do espaço explorado, destacando a eficiência relativa do algoritmo A* em comparação com a Busca de Custo Uniforme e a Busca Gulosa, em consonância com as conclusões discutidas na Seção 7.

6.2.1 Tempo de execução em função do número de cidades

O gráfico de tempo de execução evidencia o impacto do crescimento do espaço de estados à medida que o número de cidades aumenta. Observa-se que os algoritmos BFS e DFS apresentam crescimento extremamente acentuado do tempo de execução, tornando-se rapidamente inviáveis para instâncias maiores. Em particular, para $N = 12$, ambos os algoritmos ultrapassam centenas de segundos, refletindo o custo exponencial da enumeração quase completa do espaço de busca.

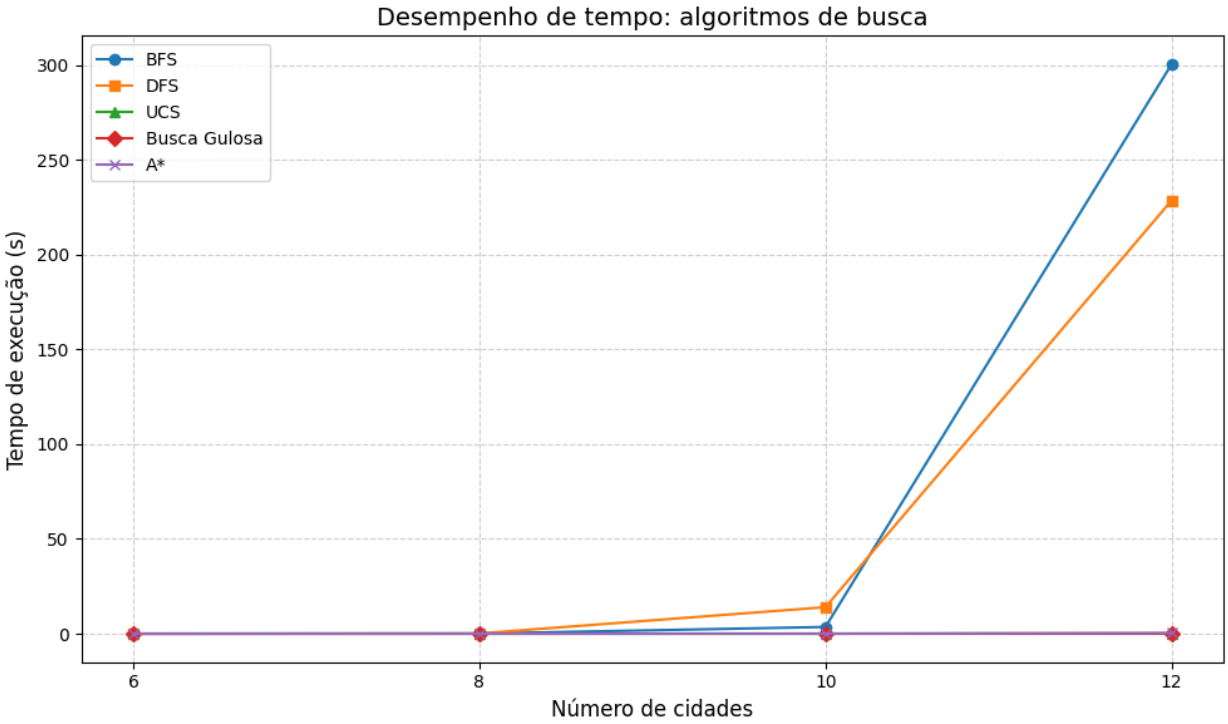

Figura 3: Tempo de execução em segundos para cada algoritmo implementado.

Esse comportamento ocorre porque ambos os algoritmos, implementados como *tree-like search* sem poda por estados visitados, exploram múltiplos caminhos diferentes que levam ao mesmo conjunto de cidades visitadas. Para $N = 12$, existem $11! = 39.916.800$ permutações possíveis de caminhos completos, e tanto BFS quanto DFS geram estados intermediários para uma grande fração dessas permutações, resultando no crescimento exponencial observado.

A Busca de Custo Uniforme (UCS) apresenta desempenho significativamente superior ao BFS e ao DFS, embora também sofra impacto com o aumento de N, devido à necessidade de explorar grande parte do espaço de estados para garantir a otimalidade.

A Busca Gulosa mantém tempos de execução praticamente constantes e muito reduzidos em todas as instâncias, resultado direto de sua estratégia míope e do pequeno número de nós expandidos. O algoritmo A*, por sua vez, apresenta um comportamento intermediário: embora mais custoso que a busca gulosa, mantém tempos de execução bastante inferiores aos da UCS, especialmente em instâncias maiores, beneficiando-se do uso de heurísticas admissíveis.

6.2.2 Número de nós gerados em função do número de cidades

O gráfico que relaciona o número de nós gerados com o número de cidades reforça as observações feitas a partir do tempo de execução. BFS e DFS apresentam crescimento explosivo no número de nós gerados, alcançando mais de 10^8 nós para instâncias com 12 cidades, o que explica sua inviabilidade prática.

A UCS, embora mais eficiente do que as buscas não informadas em termos de tempo, ainda gera um número considerável de nós, refletindo seu compromisso com a garantia de otimalidade sem auxílio heurístico.

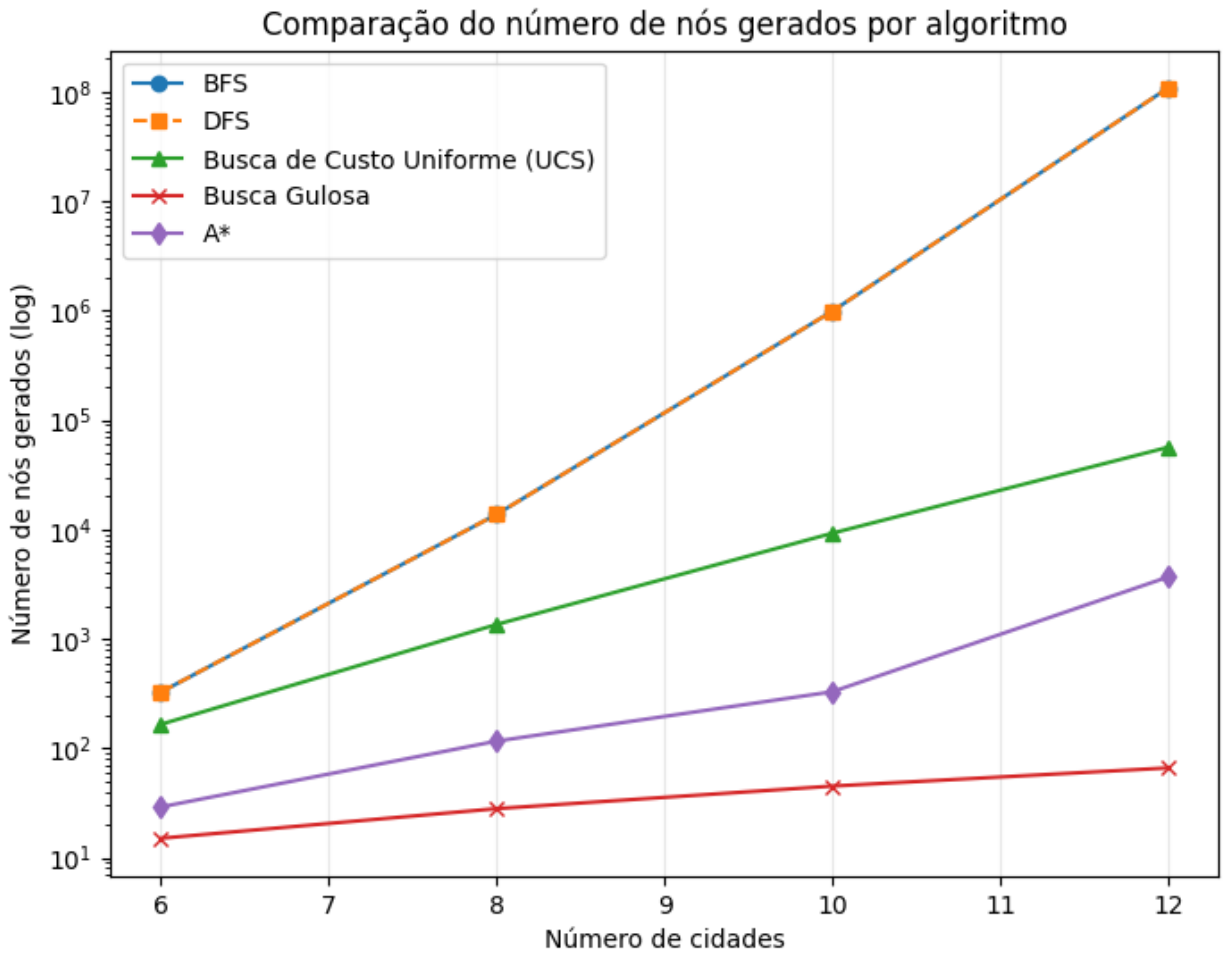

Figura 4: Nós gerados por cada algoritmo em relação a quantidade de cidades.

Em contraste, a Busca Gulosa gera um número extremamente reduzido de nós, mesmo para instâncias maiores, evidenciando sua eficiência computacional. O A* apresenta desempenho muito superior à UCS nesse aspecto, gerando ordens de magnitude menos nós, graças à capacidade da heurística baseada em árvore geradora mínima de direcionar a busca para regiões promissoras do espaço de estados.

6.2.3 Custo da solução em função do número de cidades

O gráfico de custo da solução considera apenas os algoritmos UCS, A* e Busca Gulosa, uma vez que BFS e DFS apresentaram, em todas as instâncias avaliadas, custos idênticos aos obtidos pela UCS, não acrescentando informação relevante à análise comparativa da qualidade das soluções.

Observa-se que a UCS e o algoritmo A* produzem soluções com custo idêntico em todas as instâncias consideradas, confirmando empiricamente a garantia de otimalidade de ambos os métodos. Esse resultado reforça que o uso de heurísticas admissíveis no A* não compromete a qualidade da solução, ao mesmo tempo em que reduz significativamente o esforço de busca.

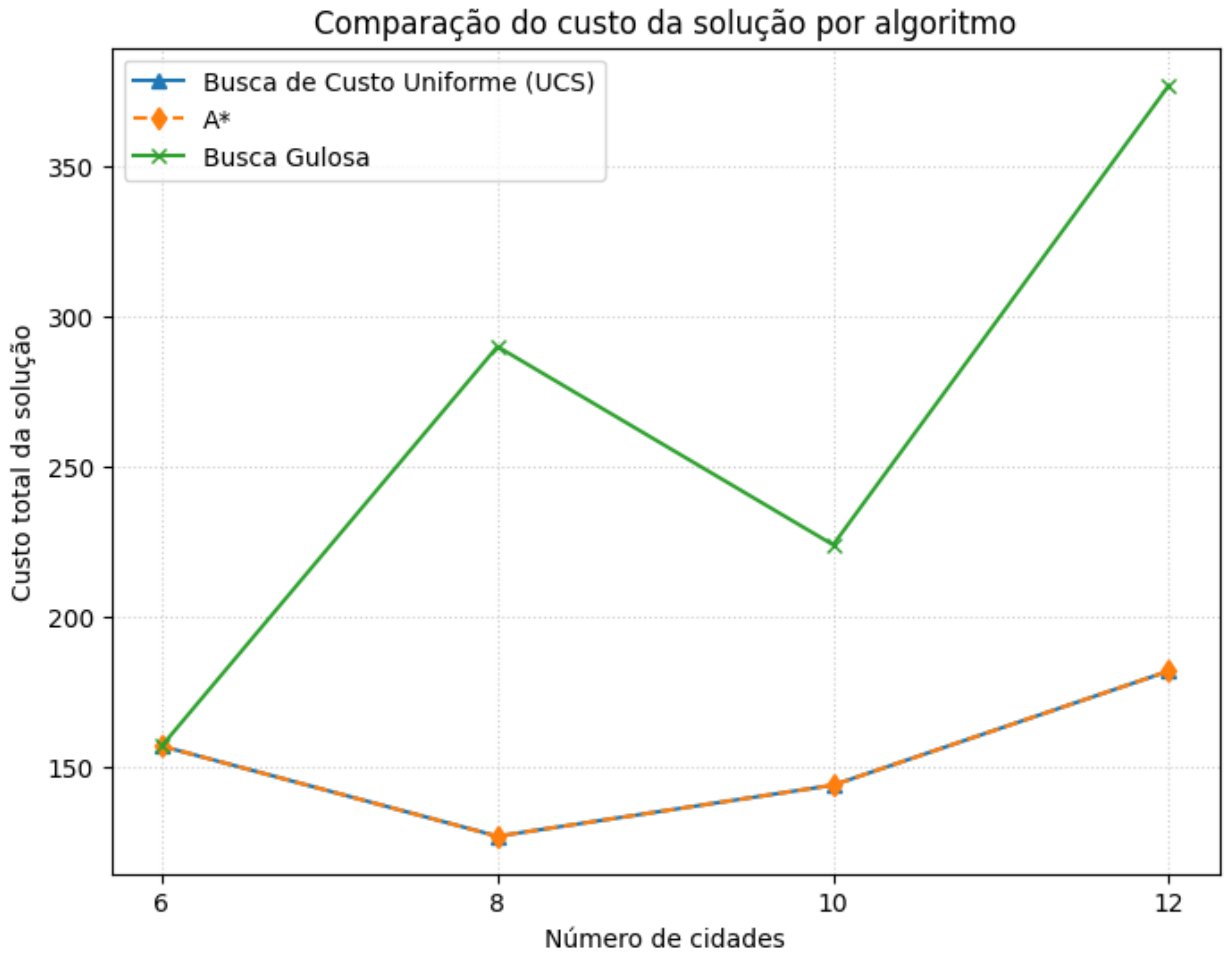

Figura 5: Comparativo do custo da solução entre os três algoritmos principais.

Por outro lado, a Busca Gulosa apresenta custos progressivamente maiores à medida que o número de cidades cresce. Para instâncias pequenas, como $N = 6$, a solução gulosa coincide com o ótimo; entretanto, para $N = 8$, 10 e 12, o custo da solução encontrada é significativamente superior ao ótimo, evidenciando o caráter local e não ótimo dessa estratégia. Esse comportamento é esperado, pois a heurística do vizinho mais próximo tende a fazer escolhas gulosas locais que podem levar a situações desfavoráveis no final do percurso, quando as cidades restantes estão distantes tanto entre si quanto da origem, forçando o algoritmo a percorrer arestas de alto custo que poderiam ter sido evitadas com planejamento mais abrangente.

Esse comportamento ilustra claramente o dilema entre eficiência computacional e qualidade da solução: a busca gulosa sacrifica otimalidade em troca de rapidez, enquanto UCS e A* preservam a qualidade da solução ao custo de maior esforço computacional.

6.2.4 Número de nós visitados em função do número de cidades

O gráfico do número de nós visitados complementa a análise do número de nós gerados, indicando o esforço efetivo de expansão realizado por cada algoritmo. BFS e DFS novamente se destacam negativamente, visitando dezenas de milhões de nós nas maiores instâncias analisadas.

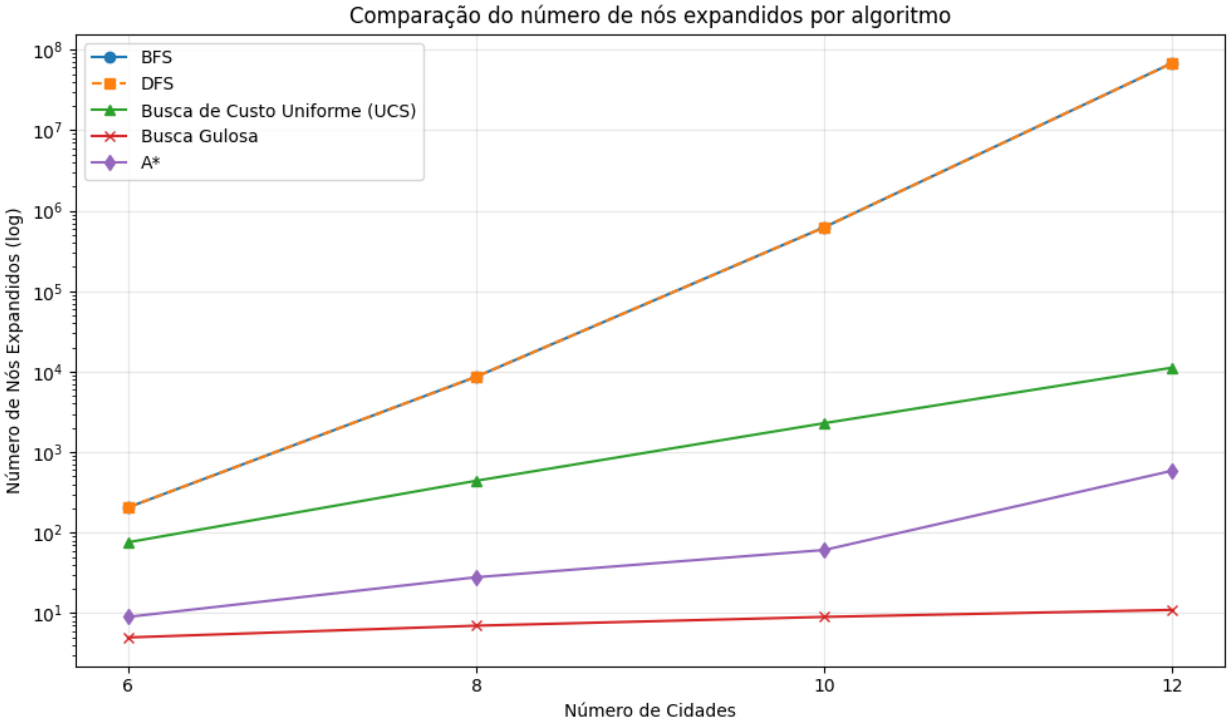

Figura 6: Nós expandidos por cada algoritmo em relação a quantidade de cidades.

A UCS apresenta uma redução expressiva no número de nós visitados quando comparada às buscas não informadas, mas ainda mantém crescimento relevante com o aumento de N. O algoritmo A* demonstra uma vantagem clara nesse critério, visitando significativamente menos nós do que a UCS, resultado direto da combinação entre custo acumulado e heurística admissível.

A Busca Gulosa, por sua vez, visita um número mínimo de nós em todas as instâncias, reforçando seu excelente desempenho computacional, embora às custas da qualidade da solução. Esses resultados confirmam que o A* oferece o melhor equilíbrio entre eficiência e otimalidade dentre os algoritmos avaliados.

7 Discussão

Os resultados apresentados nas tabelas da seção anterior permitem analisar de forma comparativa o comportamento dos algoritmos de busca aplicados ao Problema do Caixeiro Viajante, considerando tanto a qualidade das soluções obtidas quanto a eficiência computacional.

7.1 Qualidade das soluções

Em todas as instâncias analisadas, os algoritmos de Busca em Largura (BFS), Busca em Profundidade (DFS), Busca de Custo Uniforme (UCS) e A* encontraram soluções com

o mesmo custo total. Esse comportamento indica que tais algoritmos foram capazes de identificar soluções ótimas para os tamanhos de instância considerados. No caso do UCS e do A*, esse resultado é esperado, uma vez que ambos são algoritmos ótimos por construção, desde que o A* utilize heurísticas admissíveis.

A Busca Gulosa, por outro lado, apresentou soluções com custo superior ao ótimo a partir de $N = 8$ cidades. Para $N = 6$, a heurística escolhida foi suficiente para conduzir à solução ótima, porém, à medida que o número de cidades cresce, o caráter míope da estratégia gulosa torna-se evidente. Isso ocorre porque o algoritmo prioriza apenas a estimativa heurística $h(n)$, ignorando o custo acumulado $g(n)$, o que pode levar a escolhas locais que comprometem a qualidade global da solução.

7.2 Eficiência computacional

No que diz respeito à eficiência, observa-se um crescimento extremamente rápido no número de nós gerados e visitados pelos algoritmos BFS e DFS à medida que N aumenta. Para $N = 12$, ambos os algoritmos geraram mais de 100 milhões de nós, resultando em tempos de execução da ordem de centenas de segundos. Esse comportamento evidencia a explosão combinatória inerente ao espaço de estados do TSP quando estratégias não informadas são utilizadas.

Observação sobre BFS e DFS: Nota-se que ambos os algoritmos geraram e expandiram exatamente o mesmo número de nós em todas as instâncias testadas. Isso ocorre porque, implementados como *tree-like search* sem conjunto de estados explorados, ambos acabam visitando essencialmente o mesmo conjunto de estados do espaço de busca, diferindo apenas na **ordem de exploração**: DFS explora em profundidade usando uma pilha (LIFO), enquanto BFS explora por níveis usando uma fila (FIFO). Essa identidade numérica reforça que a escolha entre BFS e DFS, neste contexto, impacta apenas a ordem de descoberta das soluções, não a quantidade total de trabalho realizado.

A Busca de Custo Uniforme apresentou desempenho significativamente melhor em termos de eficiência, reduzindo drasticamente o número de nós expandidos em comparação com BFS e DFS, ao mesmo tempo em que preservou a otimalidade das soluções. Ainda assim, seu custo computacional cresce de forma considerável com o aumento do número de cidades.

O algoritmo A* destacou-se como a abordagem mais equilibrada entre qualidade da solução e eficiência computacional. Utilizando a heurística admissível baseada em Árvore Geradora Mínima (MST), o A* conseguiu encontrar soluções ótimas expandindo uma quantidade de nós substancialmente menor que o UCS, especialmente nas instâncias maiores. Isso demonstra o impacto positivo de heurísticas bem informadas na redução do espaço de busca.

7.3 Impacto das heurísticas

O papel das heurísticas mostrou-se central nos resultados obtidos. No caso da Busca Gulosa, heurísticas mais informadas reduziram drasticamente o número de nós gerados e o tempo de execução, porém sem garantir a otimalidade das soluções. Esse comportamento é claramente observado nos valores de custo mais elevados encontrados pela Busca Gulosa para instâncias com maior número de cidades.

Para o A*, a utilização de heurísticas admissíveis foi fundamental para garantir tanto a eficiência quanto a correção do algoritmo. A heurística baseada em MST forneceu estimativas mais próximas do custo real restante, permitindo que o algoritmo priorizasse estados

promissores sem violar a condição de admissibilidade. Como consequência, o A* conseguiu combinar a otimalidade do UCS com uma redução expressiva no número de expansões.

7.4 Limitações práticas

Apesar dos bons resultados obtidos por algoritmos informados, os experimentos evidenciam a limitação prática das abordagens de busca exata para o TSP à medida que o número de cidades cresce. Mesmo com heurísticas eficientes, o espaço de estados cresce de forma fatorial, tornando inviável a aplicação direta desses métodos para instâncias de grande porte.

Assim, enquanto algoritmos como UCS e A* são adequados para instâncias de tamanho pequeno a moderado, problemas reais com grande número de cidades tendem a exigir abordagens aproximadas, metaheurísticas ou técnicas de otimização específicas, que sacrificam a garantia de otimalidade em troca de viabilidade computacional.

8 Conclusão

8.1 Resumo do trabalho

Este trabalho teve como objetivo implementar, analisar e comparar diferentes algoritmos clássicos de busca aplicados ao Problema do Caixeiro Viajante (TSP), utilizando uma representação compacta de estados baseada em *bitmask*. Foram estudados algoritmos não informados (Busca em Largura, Busca em Profundidade e Busca de Custo Uniforme) e algoritmos informados (Busca Gulosa e A*), permitindo avaliar empiricamente seus comportamentos em instâncias de tamanhos variados.

Os experimentos foram conduzidos de forma controlada, utilizando sempre a mesma cidade inicial, a mesma matriz de distâncias e uma semente fixa para geração dos grafos, garantindo a comparabilidade dos resultados. As métricas analisadas incluíram o custo da melhor solução encontrada, o tempo de execução, o número de nós gerados e o número de nós visitados.

Os resultados obtidos evidenciam diferenças significativas entre as abordagens, especialmente no que diz respeito à eficiência computacional e à qualidade das soluções encontradas.

8.2 Dificuldades encontradas

Uma das principais dificuldades enfrentadas ao longo do trabalho foi o crescimento explosivo do espaço de estados do TSP, que se manifesta de forma acentuada mesmo para instâncias com número moderado de cidades. Esse fenômeno impactou diretamente os algoritmos de busca não informados, tornando inviável a execução de BFS e DFS para valores mais elevados de N sem otimizações.

Outro desafio relevante foi a definição e implementação de heurísticas adequadas para os algoritmos informados. No caso da Busca Gulosa, observou-se que heurísticas simples, embora eficientes em termos de tempo, podem conduzir a soluções de baixa qualidade. Já para o A*, a construção de heurísticas admissíveis exigiu maior cuidado conceitual e computacional, especialmente na implementação da heurística baseada em Árvore Geradora Mínima (MST).

Além disso, o gerenciamento eficiente da memória e das estruturas de dados, como filas de prioridade e conjuntos de estados visitados, mostrou-se essencial para viabilizar a execução dos experimentos e evitar redundâncias na exploração do espaço de busca.

8.3 Considerações finais

A análise realizada ao longo deste trabalho demonstra que não existe um único algoritmo que seja superior em todos os aspectos quando aplicado ao TSP. Os algoritmos BFS e DFS, apesar de conceitualmente simples, apresentam limitações práticas severas devido à explosão combinatória do problema. A Busca de Custo Uniforme garante soluções ótimas, porém com um custo computacional elevado à medida que o número de cidades aumenta.

A Busca Gulosa destacou-se pela alta eficiência, expandindo poucos nós e executando rapidamente, mas à custa da qualidade das soluções, especialmente em instâncias maiores. Por sua vez, o algoritmo A* mostrou-se a abordagem mais equilibrada entre otimalidade e eficiência, combinando a garantia de soluções ótimas com uma redução significativa no espaço de busca, desde que heurísticas admissíveis e bem informadas sejam utilizadas.

Por fim, os resultados reforçam que, embora técnicas de busca exata sejam adequadas para instâncias pequenas e médias do TSP, problemas de maior escala demandam abordagens alternativas, como algoritmos aproximados, metaheurísticas ou métodos híbridos, que oferecem soluções de boa qualidade com custo computacional viável.

Referências

[1] Russell, S., & Norvig, P. (2020). *Artificial Intelligence: A Modern Approach* (4th ed.). Pearson.